

Conception Phase

Habit Tracking App

1. Project Aim

The aim of the project is to develop the backend of a habit tracking app, using both object oriented and functional programming paradigms in Python. This will also feature a command line interface (CLI) allowing users to create, read, update and delete their daily and weekly habits.

To achieve this, a Habit Class was designed, that will act as the blueprint for every habit the user creates. The Habit Class has the following attributes and methods in the CRUD framework:

Class/ File	Function	Justification
Habit class	Individual habit object manipulation	Individual tracking
HabitManager class	Collective habit object manipulation	Manages all habit objects collectively, enabling operations that require access to multiple habits simultaneously such as finding the longest streak and filtering by periodicity
analytics.py	Analyzes habit data	Uses functional programming paradigm to analyses habit data without modifying it i.e. it uses pure functions and does not change the data.
db.py	Data persistence layer	Stores habit data beyond live sessions using SQLite.
main.py	CLI entry point	User interaction layer-provides the menu system allowing users to create, complete, view and analyse habits
test_habits.py	Unit testing	Verifies app works correctly

2. System Design

The main components of the system are the Habit Class, HabitManager, [analytics.py](#) and [main.py](#). The user interacts with the app through the CLI via [main.py](#), which provides menu options such as creating habits, viewing habits and accessing predefined habits. Each habit is created as an instance of the Habit class, collectively managed by HabitManager class. Habit object data is persisted between sessions through [db.py](#) using SQLite. Habits can also be analysed through [analytics.py](#) using functional programming principles.

3. Data Storage/ Persistence

The data storage layer of the app will use SQLite to allow habit data to persist beyond live sessions. SQLite was chosen because it is built into Python, and using it aids in learning the integration between an app and a storage layer, as databases have been studied separately - integrating this backend with SQL bridges that gap. Apart from that, SQLite is industry relevant.

What gets stored are the instance attributes of each Habit object when a habit is created, specifically: the habit name, periodicity and creation date, as well as completion records including the date and time of each completion. Data is saved to SQLite immediately when changes occur during a session ensuring no data is lost if the app closes unexpectedly.

4. Analytics Module

The analytics component ([analytics.py](#)) uses the functional programming paradigm to analyze data from the Habit objects managed by the HabitManager. It does this using pure functions that don't alter the data. It is used for analytics here to give the user insight into their habits when queried, and to serve that function for a habit tracker - showing a user their progress or opportunities for improvement.

The following four functions are implemented:

- `get_all_habits()` - returns a list of all currently tracked habits
- `get_habits_by_periodicity()` - returns a list of habits filtered by periodicity (e.g. all daily habits or all weekly habits)
- `get_longest_streak_overall()` - returns the longest streak across all defined habits
- `get_longest_streak_for_habit()` - returns the longest streak for a specific given habit.

It is important to note the distinction between three types of functions in this system:

Type	Location	Purpose
Habit Instance methods	habit.py	Work on a single Habit object's own data using self-e.g. <code>mark_complete()</code> , <code>get_current_streak()</code>
HabitManager instance methods	<code>habit_manager.py</code>	Manage the collective Habit objects e.g. <code>add_habit()</code> , <code>delete_habit()</code>
Analytics functions	analytics.py	Pure functions that receive Habit data and return results without changing it.

5. Tools

Tool	Version/ library	Justification
Programming language	Python 3.11.7	-Project requirement and Python offers the datetime and sqlite3 libraries eliminating the need for installation. -Large community and documentation
Built-in libraries	• datetime-handling dates for completion tracking • sqlite3 – database storage	-Essential for tracking when habits are created and completed -datetime handles all date and time operations -sqlite3 is perfect for a single-user app like a habit tracker
External libraries	Pytest - unit testing	- The only external library currently required is pytest for unit testing, as recommended by the project brief. -All other functionality is handled by Python's built-in libraries, keeping the project lightweight with minimal dependencies
For CLI	Python's built in <code>input()</code> function	Additional libraries such as <code>questionary</code> may be introduced during the development phase to enhance the CLI interface if needed.

6. User Interaction/Flow

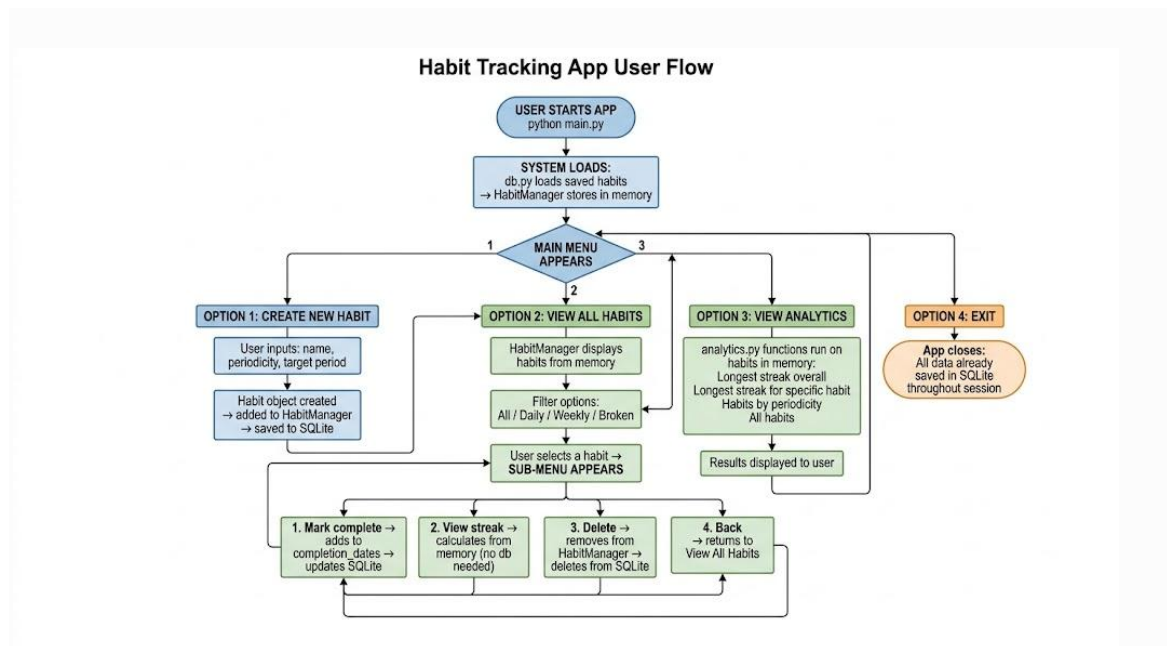


Figure 1: Habit Tracker App User Flow. Diagram generated using AI (Gemini) based on the system design developed for this project.

7. Predefined Habits

Habit	Periodicity	Target Period
Run	Daily	7 days
Python coding	Daily	14 days
Church	Weekly	4 weeks
Letter to future self	Monthly	6 months
Read daily	Daily	30 days

8. UML Class Diagram

